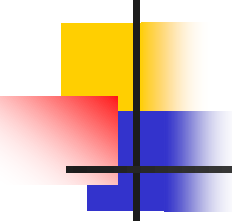


Software Engineering Week 7,8

INSTRUCTOR : DR. NATASH ALI MIAN

School of Computer and Information Technology

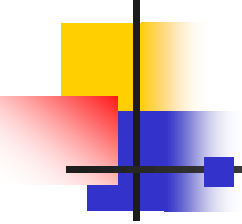
Beaconhouse National University



What is UML and Why we use UML?

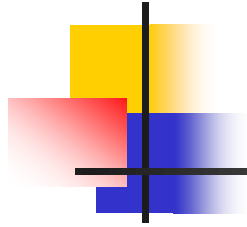
- Why we use UML?
 - Use graphical notation: more clearly than natural language (imprecise) and code (too detailed).
 - Help acquire an overall view of a system.
 - UML is *not* dependent on any one language or technology.
 - UML moves us from fragmentation to standardization.

Use Case

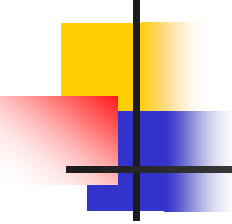


■ Use Case

- (Representation of a Single requirement in a diagram form)
- Use Case Diagram
 - Collection of a few requirements/ collection of a few use cases
- Use Case Model
 - A complete representation of all users/actors and their requirements

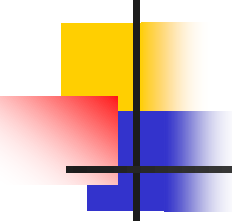


- **Actors** (User of the system)
- **Link** (line)
- **Use Case** (Feature)
- System Boundary



1

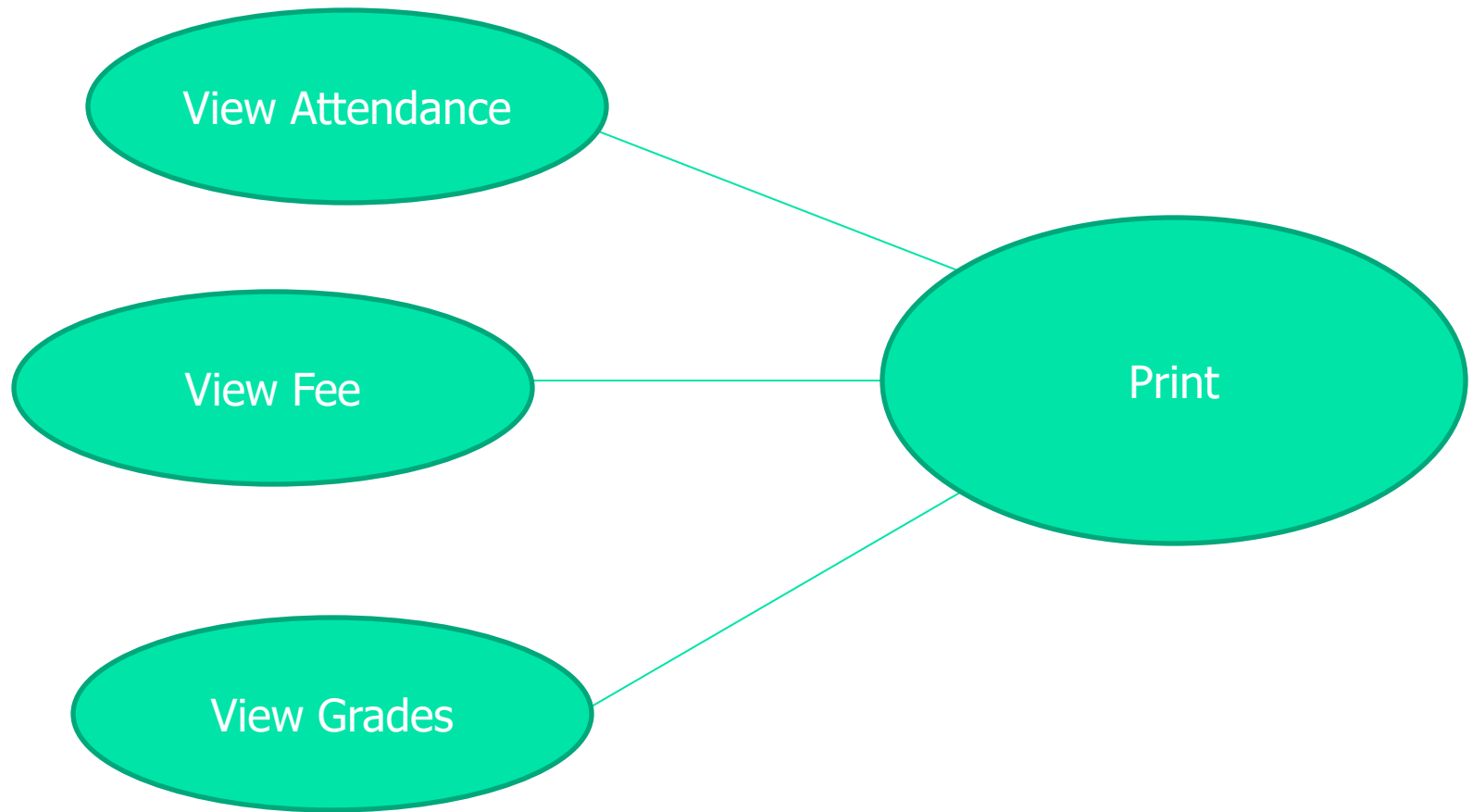
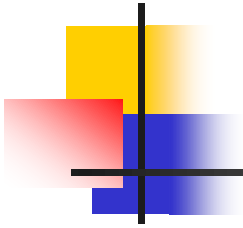
- **View Attendance**
 - **View Fee**
 - View Result
 - View Academic History
 - View Enrolment
 - **Print**
-
- **View Attendance and then print it**



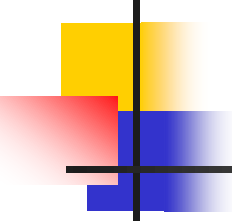
2

- **View my fee**

- **View my fee and Print**
- **View Attendance**
- **View Attendance and Print**
- **View Attendance &**
- **Print Attendance**
- View Result and Print
- View Academic History and Print
- View Enrolment and Print



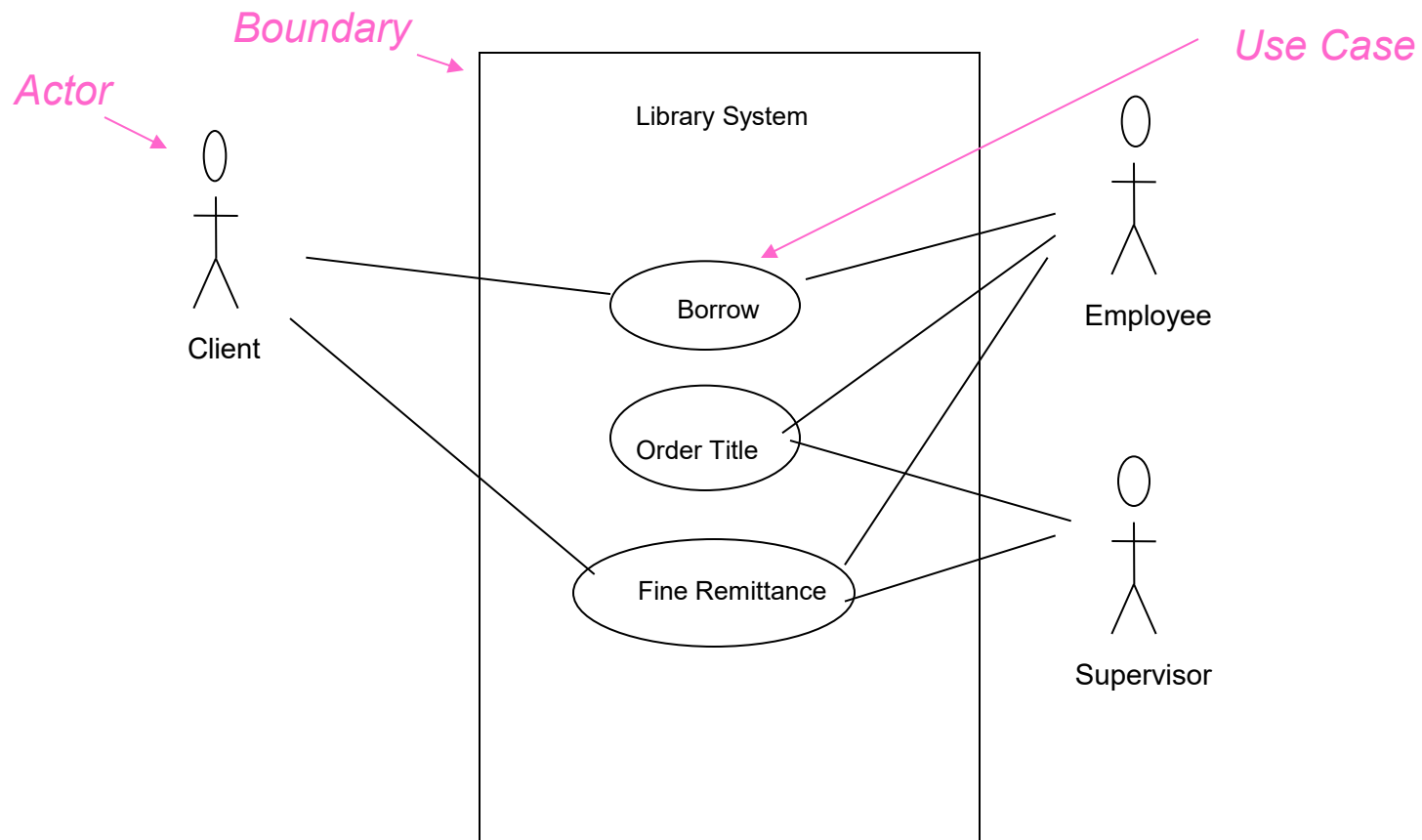
- View Attendance
- Print Attendance
- View Result
- Print Result
- View Academic History
- Print Academic History
- View Enrolment
- Print Enrolment



Use-Case Diagrams

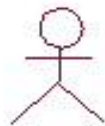
- A use-case diagram is a set of use cases
- A use case is a model of the interaction between
 - External users of a software product (actors) and
 - The software product itself
 - More precisely, an actor is a user playing a specific role
- describing a set of user **scenarios**
- capturing user requirements
- **contract** between end user and software developers

Use-Case Diagrams

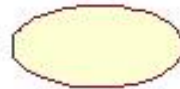


Use-Case Diagrams

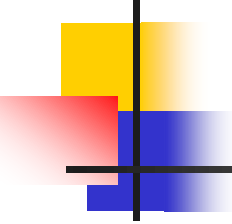
- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.
- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.
- **System boundary:** rectangle diagram representing the boundary between the actors and the system.



Actor



Use Case



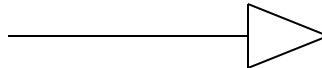
Use-Case Diagrams

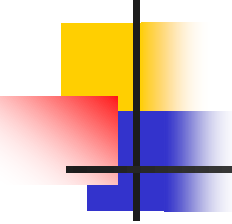
- Association:

communication between an actor and a use case; Represented by a solid line.



- Generalization: relationship between one general use case and a special use case (used for defining special alternatives) Represented by a line with a triangular arrow head toward the parent use case.

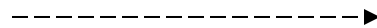




Use-Case Diagrams

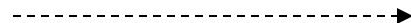
Include: a dotted line labeled <<include>> beginning at base use case and ending with an arrow pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use “include” in stead of copying the description of that behavior.

<<include>>



Extend: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares “extension points”.

<<extend>>



Use-Case Diagrams

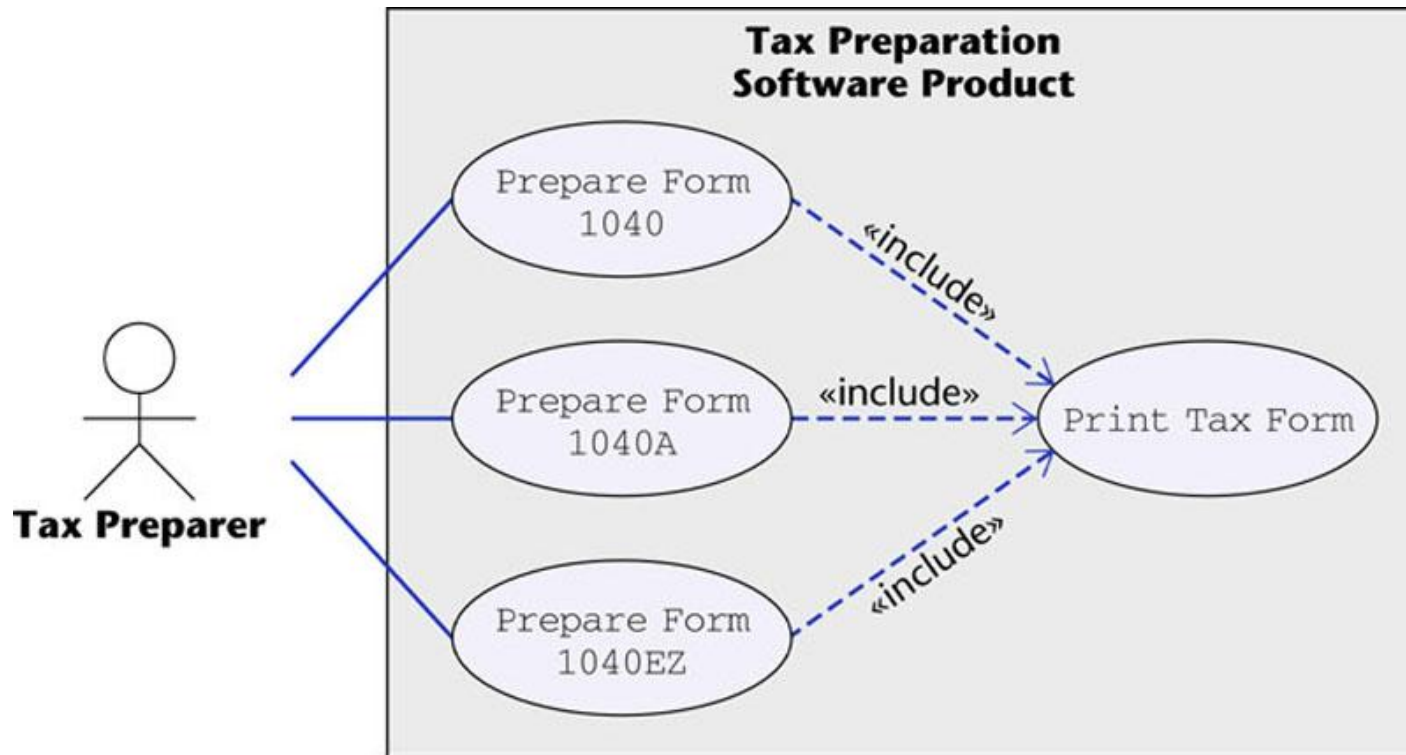
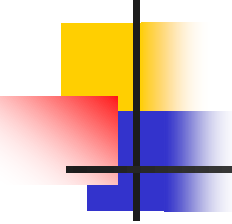


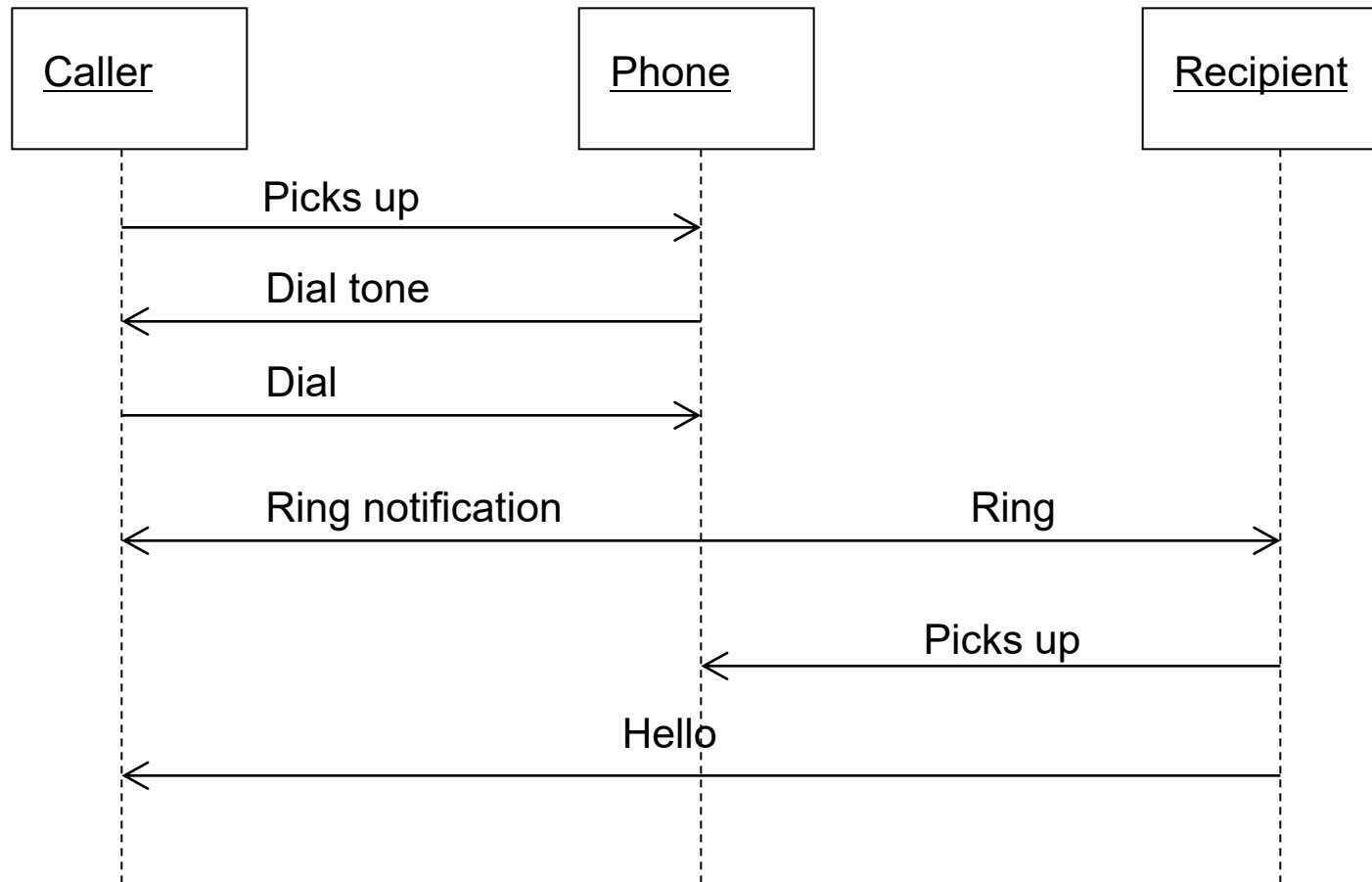
Figure 16.12



Interaction Diagrams

- show how objects interact with one another
- UML supports two types of interaction diagrams
 - Sequence diagrams
 - Collaboration diagrams

Sequence Diagram(make a phone call)



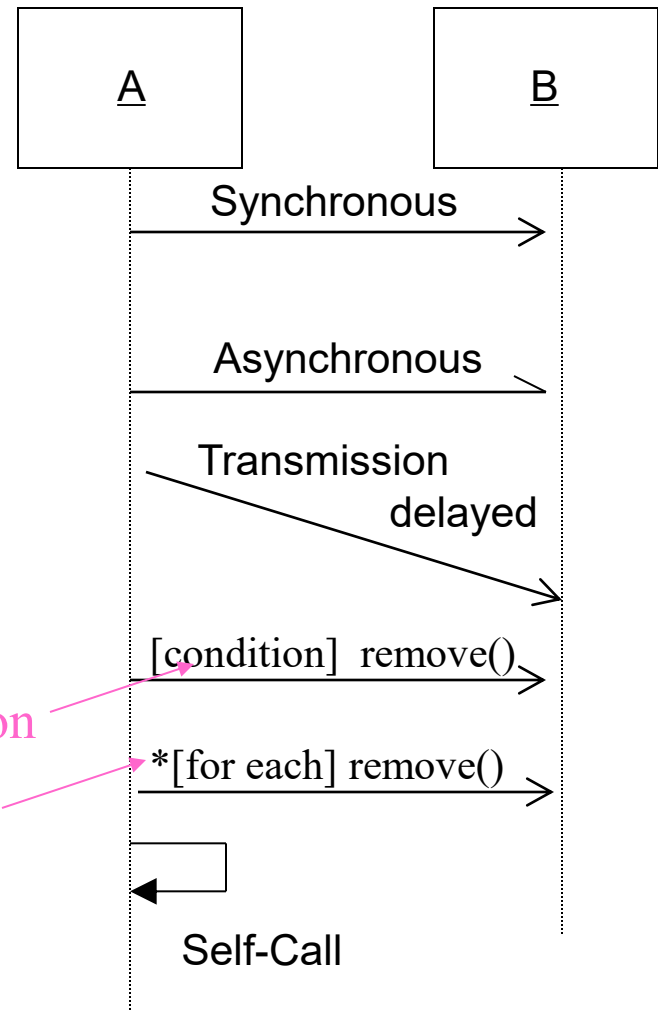
Sequence Diagram: Object interaction

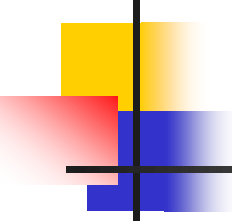
Self-Call: A message that an Object sends to itself.

Condition: indicates when a message is sent. The message is sent only if the condition is true.

Condition

Iteration





```
Void main ()
```

```
{
```

```
Int x;
```

```
if (expression)
```

```
{
```

```
int a ;
```

```
}
```

```
}
```

variable can be used/ accessed - **SCOPE**

Existence of variable in the memory – **LIFETIME**

Sequence Diagrams – Object Life Spans

■ Creation

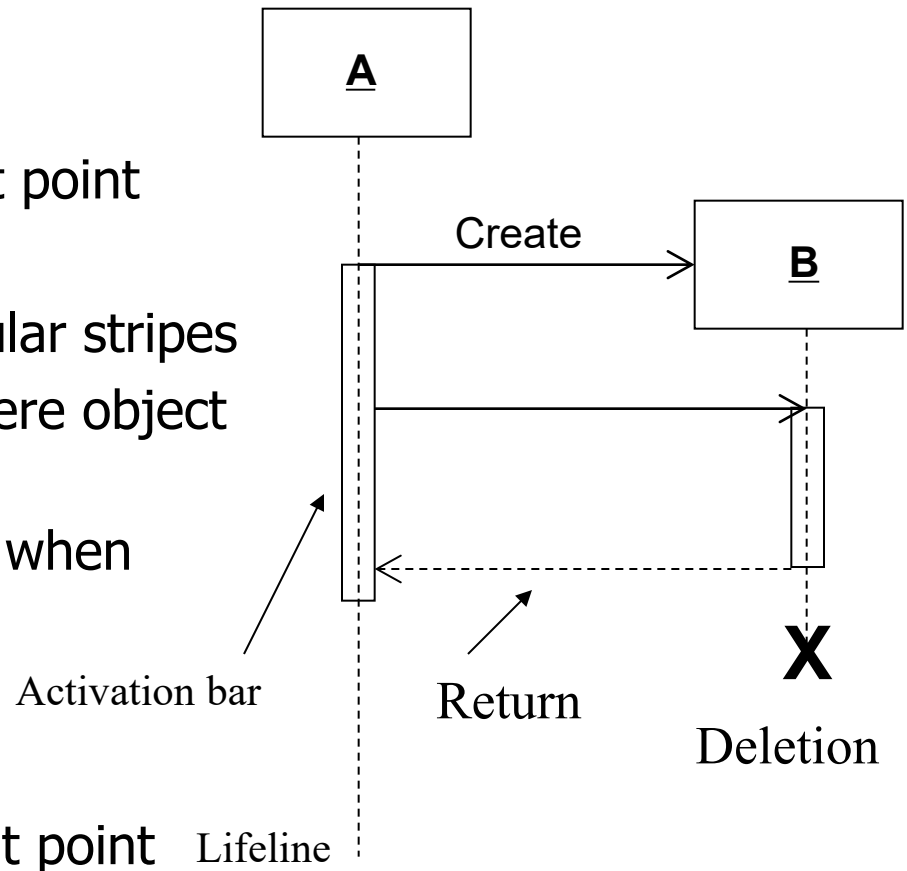
- Create message
- Object life starts at that point

■ Activation

- Symbolized by rectangular stripes
- Place on the lifeline where object is activated.
- Rectangle also denotes when object is deactivated.

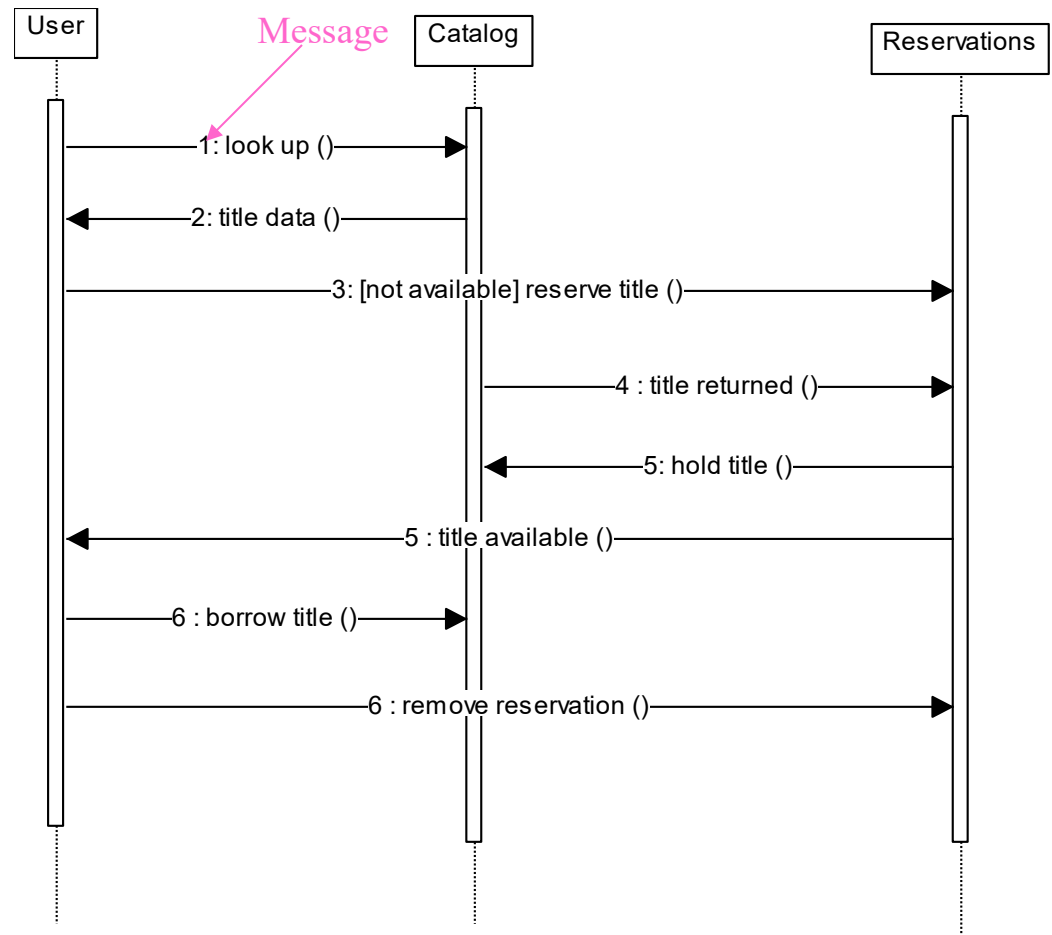
■ Deletion

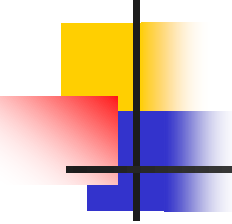
- Placing an 'X' on lifeline
- Object's life ends at that point



Sequence Diagram

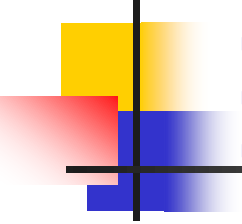
- Sequence diagrams demonstrate the behavior of objects in a use case by describing the objects and the messages they pass.
- The horizontal dimension shows the objects participating in the interaction.
- The vertical arrangement of messages indicates their order.
- The labels may contain the seq. # to indicate concurrency.





Class Diagram

- Object Oriented Design
- Set of Classes (interacting components)
- Modules, Components, Classes



■ Class Name

- Possible Attributes
 - Possible Operations/functions
 - Relationship with : A ()
 - Relationship with : B ()
-

- Real life objects
- Generalized Class

Class Name

Set of Variables

Set of Function

Access Specifier

Private

Protected

Class A

Class B is a child of class A

(It will inherit/access all

functions of class A that are either

public or **protected**)

Public

private:

Int a;

int b

int c

public:

Class SE_Test

PR int a;

PR

PT

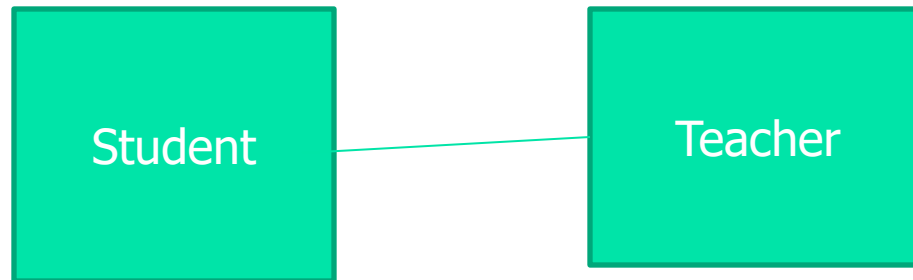
PU int sum (int a, int b);

- Cardinality

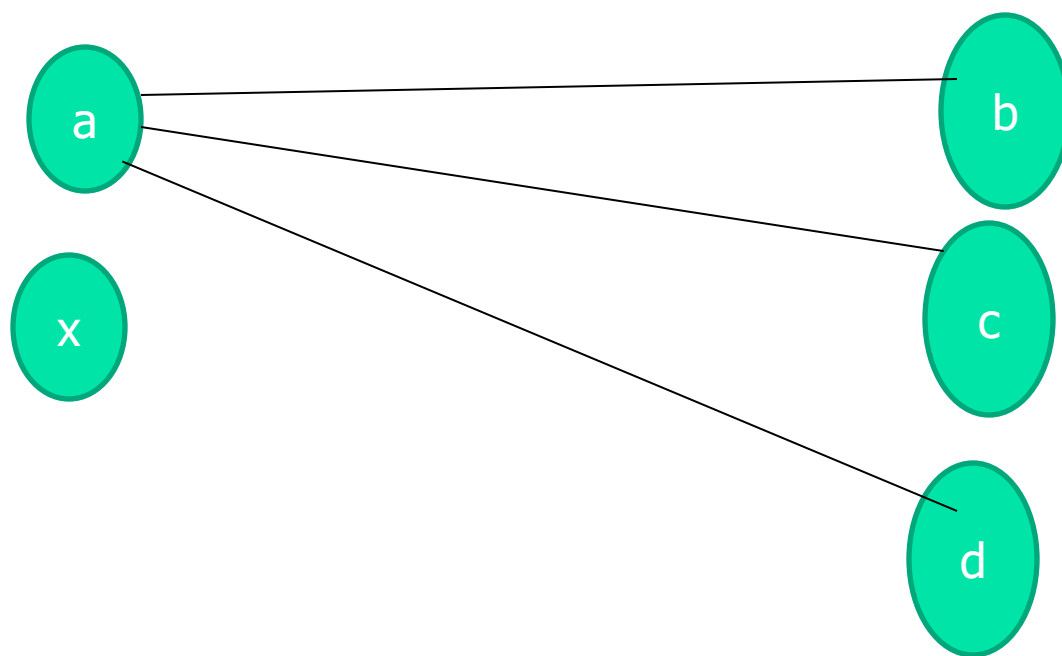
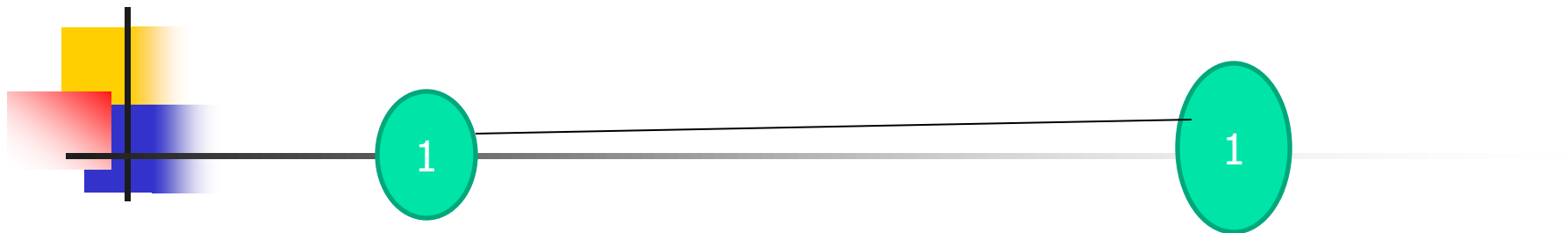
- 1 – 1

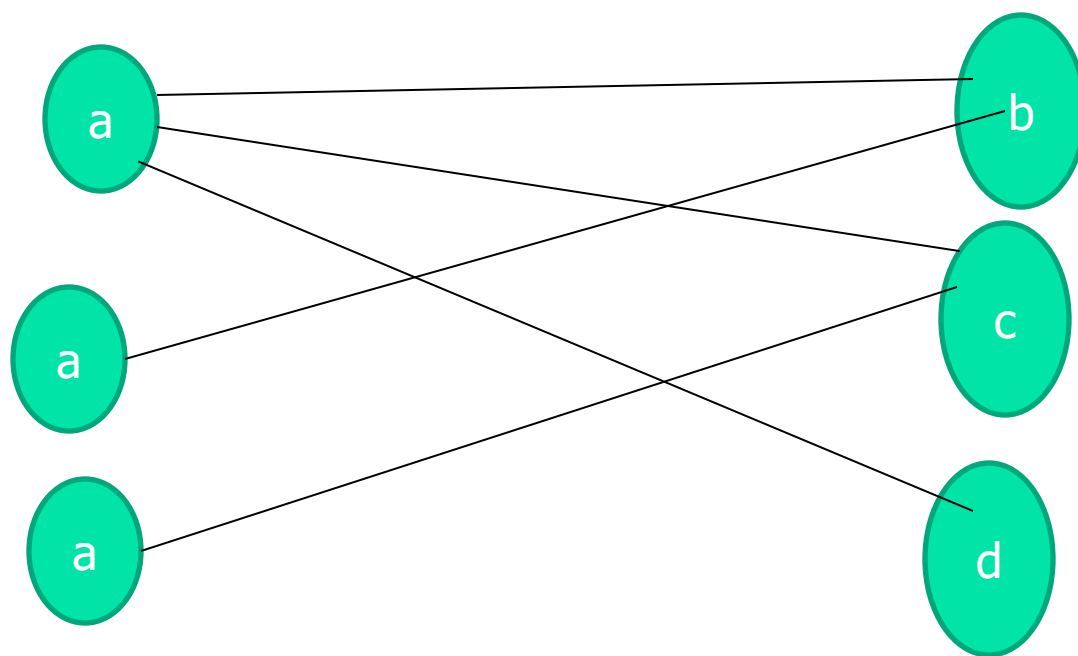
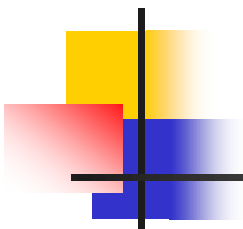
- 1 – N

- M – N



- One Teacher teaches how many students? MANY
- One student is taught by how many teachers? MANY





Class – Structure

Abstract Data Type / User Defined Type

MasterClass

Var_1 (data type, access specifier)
Var 2 (data type, access specifier)
Var 3 (data type, access specifier)
Var 4 (data type, access specifier)

Func 1 ()
Func 2 ()
Func 3 ()
Func 4 ()

ClassB

int Var5
float Var6
MASTERCLASS var7

Func a ()
Func b ()

ClassB sends the call (function call) to Masterclaass with parameters (function parameters). The instruction (function call) is to add two numbers. INST1 (a,b), result will be the sum of a and b.

INT

MASTERCLASS

CLASSB

int a,b,c,d;

MASTERCLASS b;

Primitive Data Type

Class defines the type of variables and the functions that can be performed.

Physical Instance of the class - > (Object)

Object is the **physical instance** of the class.

b

Var 1 =0

Var 2 =0

Var 3 =0

Var 4 =0

Func 1 ()

Func 2 ()

Func 3 ()

Func 4 ()

Var7

Var 1 =0

Var 2 =0

Var 3 =0

Var 4 =0

Func 1 ()

Func 2 ()

Func 3 ()

Func 4 ()

Obj

Var5= 10

Var6 =0

var7

Func a ()

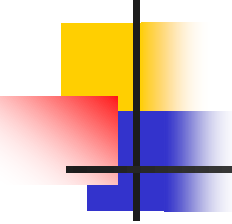
Func b ()

MASTERCLASS b,var7;

Var7.func1(1,3);

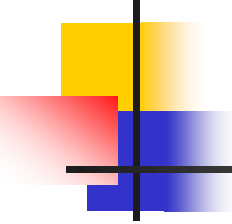
Obj.func4(1,2,2,4,4,4);

Obj.var1= 35;



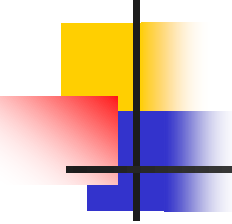
Class diagram

- A class diagram depicts classes and their interrelationships
- Used for describing **structure and behavior** in the use cases
- Provide a conceptual model of the system in terms of entities and their relationships
- Used for requirement capture, end-user interaction
- Detailed class diagrams are used for developers

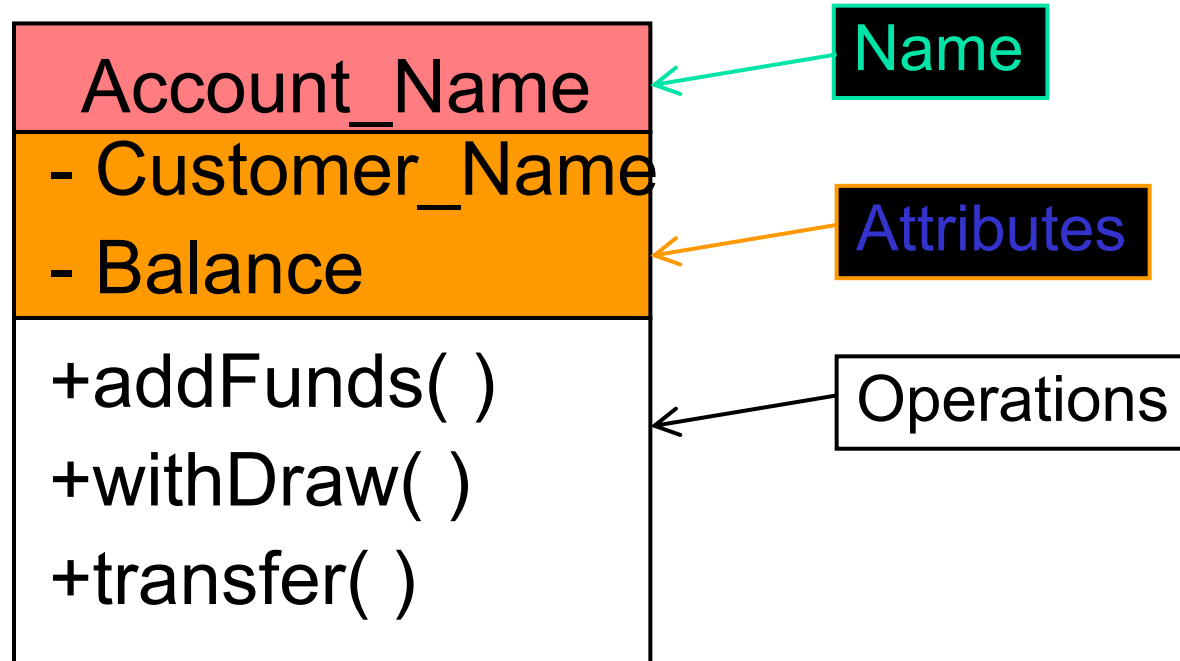


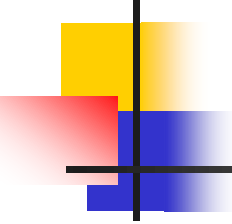
Class diagram

- Each class is represented by a rectangle subdivided into three compartments
 - Name
 - Attributes
 - Operations
- Modifiers are used to indicate visibility of attributes and operations.
 - '+' is used to denote *Public* visibility (everyone)
 - '#' is used to denote *Protected* visibility (friends and derived)
 - '-' is used to denote *Private* visibility (no one)
- By default, attributes are hidden and operations are visible.



Class diagram

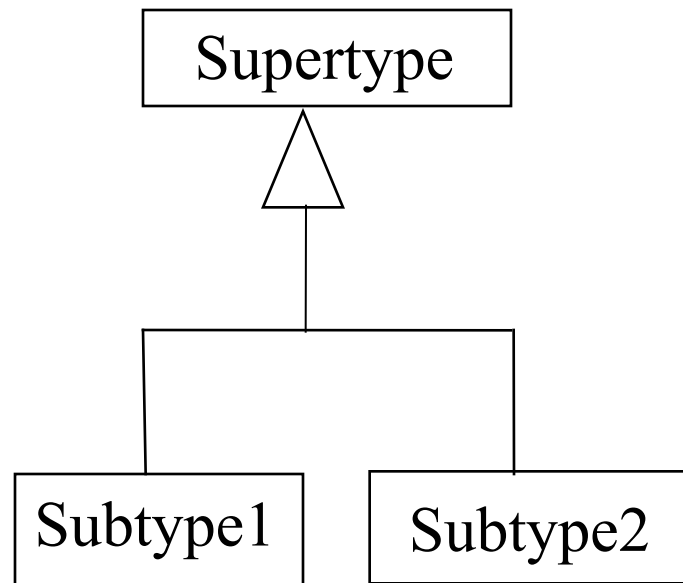




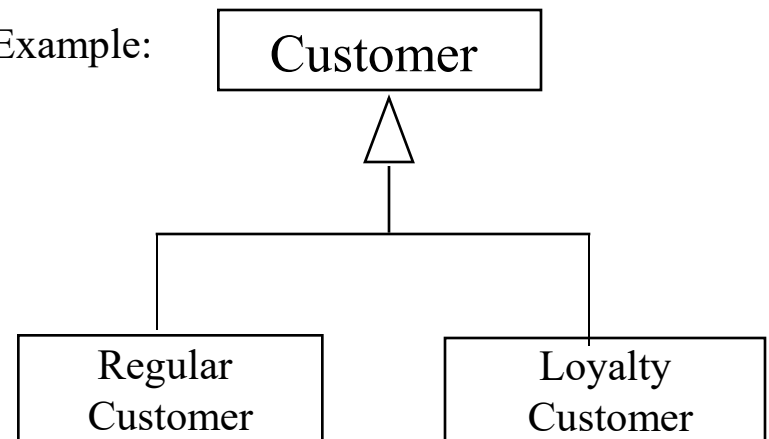
OO Relationships

- There are two kinds of Relationships
 - Generalization (parent-child relationship)
 - Association (student enrolls in course)
- Associations can be further classified as
 - Aggregation
 - Composition

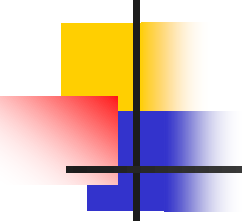
OO Relationships: Generalization



Example:



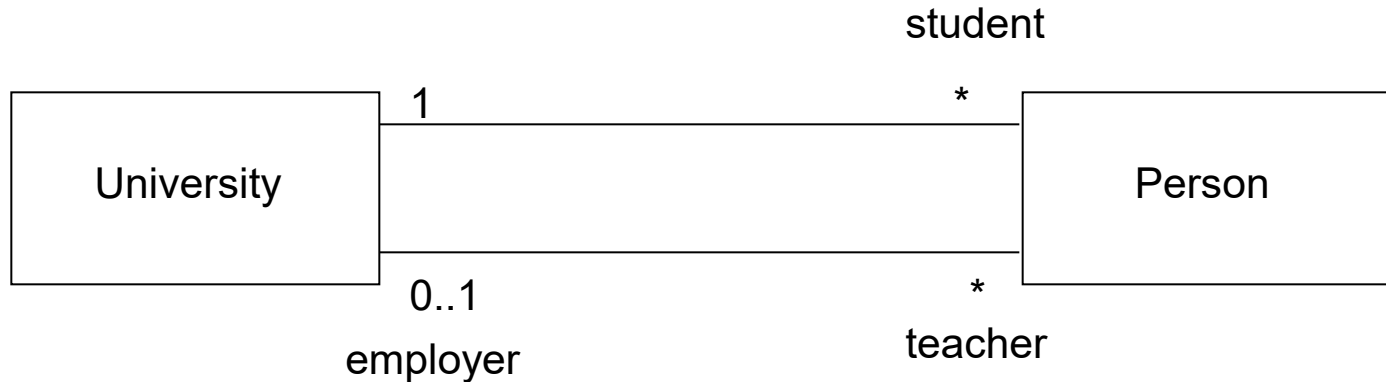
- Inheritance is a required feature of object orientation
- Generalization expresses a parent/child relationship among related classes.
- Used for abstracting details in several layers



OO Relationships: Association

- Represent relationship between instances of classes
 - Student enrolls in a course
 - Courses have students
 - Courses have exams
 - Etc.
- Association has two ends
 - Role names (e.g. enrolls)
 - Multiplicity (e.g. One course can have many students)
 - Navigability (unidirectional, bidirectional)

Association: Multiplicity and Roles

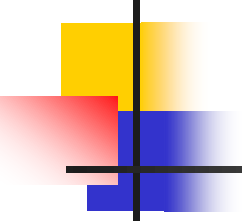


Multiplicity

Symbol	Meaning
1	One and only one
0..1	Zero or one
M..N	From M to N (natural language)
*	From zero to any positive integer
0..*	From zero to any positive integer
1..*	From one to any positive integer

Role

"A given university groups many people; some act as students, others as teachers. A given student belongs to a single university; a given teacher may or may not be working for the university at a particular time."

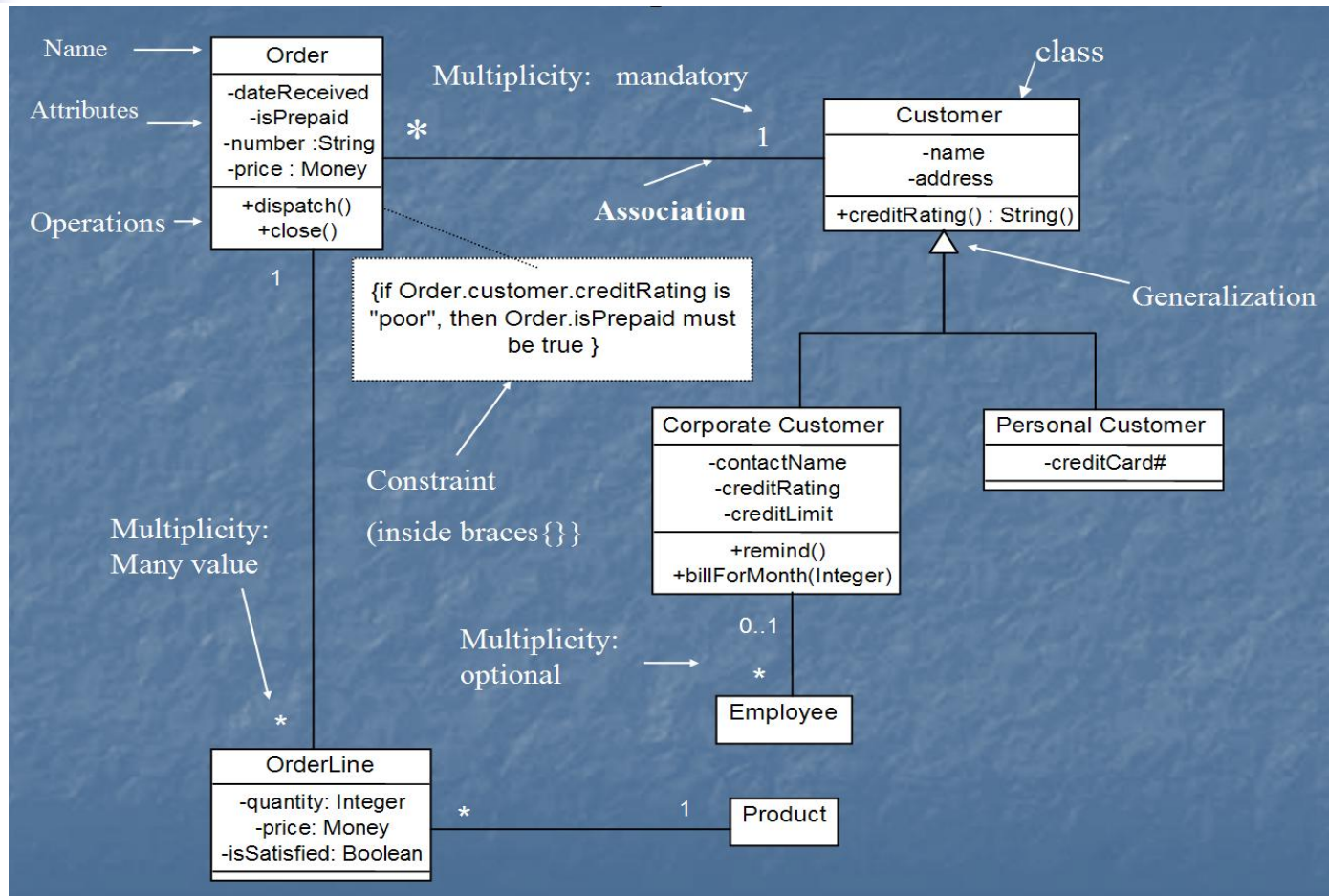


1 – M 1 - * (Zero to Many)

1 – 5 (**One to many**), min one and maximum five

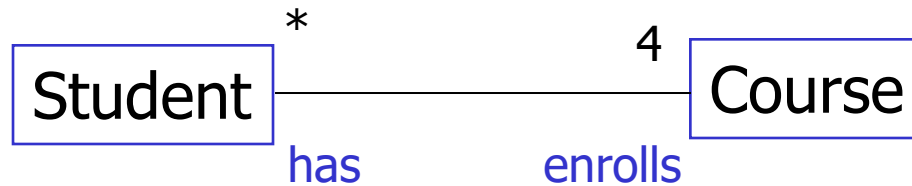
10 – 50 (Many to Many), minimum ten and maximum 50

Class diagram



[from *UML Distilled Third Edition*]

Association: Model to Implementation

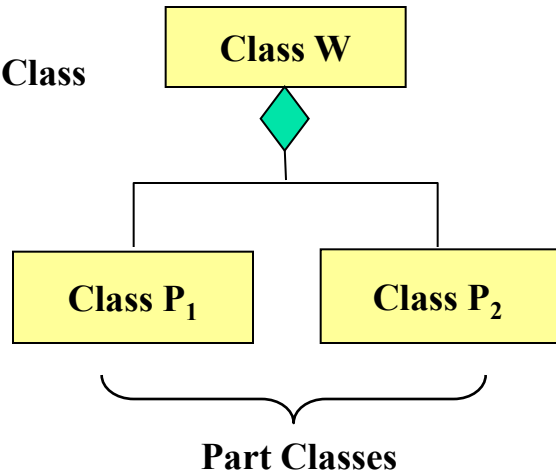


```
Class Student {  
    Course enrolls[4];  
}
```

```
Class Course {  
    Student have[];  
}
```

OO Relationships: **Composition**

Whole Class



[From Dr.David A. Workman]

Example

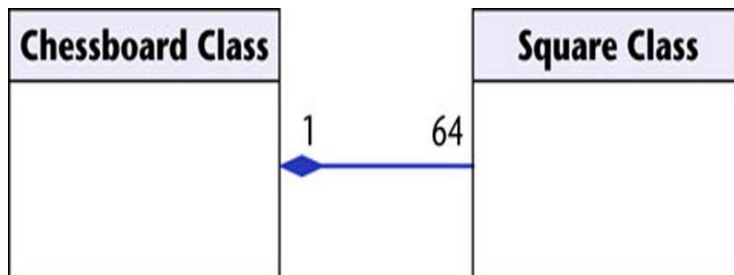


Figure 16.7

Association

Models the part–whole relationship

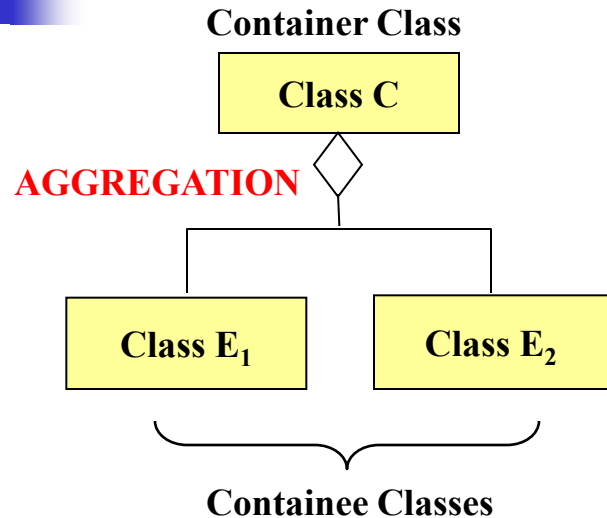
Composition

Also models the part–whole relationship but, in addition, Every part may belong to only one whole, and If the whole is deleted, so are the parts

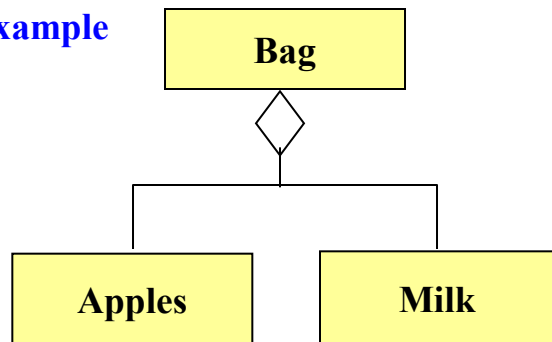
Example:

A number of different chess boards: Each square belongs to only one board. If a chess board is thrown away, all 64 squares on that board go as well.

OO Relationships: Aggregation



Example

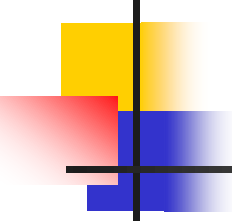


Aggregation:

expresses a relationship among instances of related classes. It is a specific kind of Container-Containee relationship.

express a more informal relationship than composition expresses.

Aggregation is appropriate when Container and Containees have no special access privileges to each other.



Aggregation vs. Composition

■ **Composition** is really a strong form of **association**

- components have only one owner
- components cannot exist independent of their owner
- components live or die with their owner
- e.g. Each car has an engine that can not be shared with other cars.

■ **Aggregations**

may form "part of" the association, but may not be essential to it. They may also exist independent of the aggregate. e.g. Apples may exist independent of the bag.